01 · Sensor fusion

Data fusion: RGB Camera + LiDAR 3D

Alessandro La Mantia

Person detection and depth estimation on the KITTI Raw dataset.

Python · NumPy · OpenCV · Streamlit · Open3D

Detecting people and estimating their depth

The goal is to **combine RGB camera images with 3D LiDAR measurements** to detect people in the scene and estimate their depth.

In general, the project involves:

- Observing a scene through **images and point clouds**.
- **Relating information** from the two sensors.
- Showing detections and estimated depth in an **interactive demo**.

Why is Data Fusion needed?

People can be detected using **RGB alone or LiDAR alone**, with a suitable algorithm. Data fusion adds information and compensates for some limitations of the individual sensors.

Sensor	What it provides	Main limitation
RGB camera	Colours, texture and shape: useful for recognising a person.	Real distance cannot be obtained directly from a single image.
LiDAR	3D positions and distances of measured points.	Recognition requires interpreting the shape of the cloud, which may contain few points on the object.
RGB + LiDAR	Visual recognition linked to a depth position.	Requires calibration and matching acquisitions.

In this project: HOG detects people in the RGB image; fusion associates the boxes with depth estimated using LiDAR.

The data used: 32 KITTI frames

The project uses a subset of the **KITTI Raw dataset**, recorded by a moving vehicle with an RGB camera and a LiDAR.

The **32 frames** are 32 acquisitions from the same sequence at different times. Each acquisition includes:

- **An RGB image** showing the scene captured by the camera.
- **A LiDAR scan**, i.e. the 3D point cloud associated with the image.
- **A shared index** that allows the two files to be matched.

We therefore work with **32 image + point cloud pairs**: as the vehicle moves forward, the observed scene changes.

A modular pipeline

Module	Task
calibration.py	Transformations between sensors
projection.py	Projection and filtering
detection_fusion.py	**People and depth:** HOG detects people in the image; the module selects LiDAR points projected inside the boxes and estimates depth using an IQR filter and the median.
fusion.py	**Coloured 3D cloud:** assigns each LiDAR point its corresponding RGB pixel colour and exports coordinates and colours to a PLY file.
visualization.py	2D and 3D visualisation
streamlit_app.py	Local interface

From LiDAR to camera

LiDAR: X forward, Y left, Z up.
Camera: X right, Y down, Z forward.

> $P = [X, Y, Z, 1]^T$
> $P_{cam} = T_velo_to_cam \cdot P$

The calibration matrix **T_velo_to_cam** combines **rotation** of the axes and **translation** between the origins of the sensors, which are mounted in different positions.

The physical point stays the same: its coordinates change as we move from the LiDAR reference frame to the camera reference frame.

The additional value 1 allows rotation and translation to be applied with a single 4×4 matrix.

Composing the transformation

After changing reference frames, we must **link the 3D LiDAR points to the RGB image pixels**.

Matrix **M** combines the change of reference frame, rectification and Camera 2 projection into a single operation.

> $M = P_rect_02 \cdot R_rect_00 \cdot T_velo_to_cam$

The transformations are applied **from right to left**:

- **T_velo_to_cam:** expresses LiDAR points in the camera reference frame.
- **R_rect_00:** aligns the reference frame with the rectified images.
- **P_rect_02:** applies the Camera 2 projection.

From 3D points to pixels

For each LiDAR point, we calculate **where it appears in the image**. Coordinates **u** and **v** indicate the pixel **column** (horizontal) and **row** (vertical), respectively, starting from the top left corner.

We apply matrix M to the 3D point and obtain three intermediate values: a, b and d.

$$[a, b, d]^T = M \cdot [X, Y, Z, 1]^T$$
$$u = a / d \quad v = b / d$$

Dividing by d produces **perspective**: for the same lateral displacement, more distant points appear closer to the centre of projection.

Example: (u, v) = (400, 150) means column 400 and row 150. We can now associate that 3D point with a pixel in the image.

From the person in the image to LiDAR points

1. HOG describes shape. HOG (Histogram of Oriented Gradients) summarises the directions of brightness changes: it highlights edges and contours, useful for describing a person’s silhouette.

2. SVM classifies. A pretrained linear SVM (Support Vector Machine) evaluates these features and decides whether an image region resembles a person. The search covers regions of different sizes.

3. Boxes are obtained. Detected regions are enclosed in rectangles; NMS reduces duplicate rectangles around the same subject.

4. LiDAR measurements are associated. For each box, we select the points that fall inside it after projection. Their depths are used to estimate the subject’s depth.

A box may also contain road and background: not all points belong to the person. The detector score is not a probability.

IQR filter and median

Points inside a box can have very different depths, partly because of the background. We want an estimate that is **less sensitive to outliers**.

- **IQR filter:** considers the middle half of the values. Q1 and Q3 are its limits; IQR = Q3 – Q1. We discard values outside [Q1 – 1.5 IQR; Q3 + 1.5 IQR].
- **Median:** after filtering, we take the middle value of the sorted depths; if there is an even number of values, we average the two middle values.

> Depth: 10, 10, 11, 11, 100 m
> Q1 = 10, Q3 = 11 → range: 8.5–12.5 m
> Discard 100 m → median = (10 + 11) / 2 = 10.5 m

The Streamlit demo

Choose the RGB image

View the LiDAR

Click “Find people”

The result shows boxes, depth and a table with points in the box, points used and HOG score.

Open the local app

Challenges and solutions adopted

Challenge	How it was addressed
Aligning sensors with different axes and positions.	Using KITTI calibration and composing the reference frame change, rectification and projection.
Estimating depth with background points in the boxes and sparse LiDAR measurements.	IQR filtering to reject depth outliers and the median of the remaining points.

What I learned

1. **Calibrating and connecting different sensors.**
   I learned to transform points from the LiDAR reference frame to the camera reference frame and project them onto image pixels.

2. **Combining RGB images and LiDAR measurements.**
   I learned to associate 3D points with the boxes of people detected in the image, combining visual recognition with depth measurements.

3. **Estimating depth and understanding the limitations.**
   I learned to use IQR filtering and the median, and to recognise the effects of sparse points, background inside boxes and detector false positives.

Concrete changes to improve results

What to change	Expected result
Replace HOG + SVM with a pretrained detector, such as YOLO, selecting the person class.	Improve detection of small or partially occluded people and reduce false positives.
Match the same person across consecutive frames and filter their depth estimates over time.	Reduce fluctuations in the displayed depth from one frame to the next.

thank you for your attention